EECE7352 Computer Architecture

Homework 1

Sakthivel P. Sivakumar

NUID: 002312294

Part D:

Optimization Results

X86_64

-O0 optimization

```
Trying 50000000 runs:
Microseconds for one run through Dhrystone:      0.12
Dhrystones per Second:                        8460237
```

-O3 optimization

```
 Trying 500000000 runs:
 Microseconds for one run through Dhrystone:      0.03
 Dhrystones per Second:                        31328320
```

ARM_V8

-O0 optimization

```
Trying 50000000 runs:
Microseconds for one run through Dhrystone:      0.19
Dhrystones per Second:                        5313496

sakthivelps@degrees:~/EECE7352/sakthivel/HW1/PartD/Arm_v8$
```

-O3 optimization

```
Trying 50000000 runs:
Microseconds for one run through Dhrystone:      0.05
Dhrystones per Second:                       19685040
```

Assembly Snippet

X_86 (O3) Proc_7:

```
Proc_7:
.LFB23:
    .cfi_startproc
# dry.c:907:    *Int_Par_Ref = Int_2_Par_Val + Int_Loc;
    lea eax, [rdi+2+rsi]    # tmp104,
    mov DWORD PTR [rdx], eax    # *Int_Par_Ref_6(D), tmp104
# dry.c:908: } /* Proc_7 */
    ret
    .cfi_endproc
.LFE23:
    .size   Proc_7, .-Proc_7
    .p2align 4
    .globl  Proc_8
    .type   Proc_8, @function
```

Explanation:

Different Registers:

- **rdi** = first argument (Int_1_Par_Val)
- **rsi** = second argument (Int_2_Par_Val)
- **rdx** = third argument (Int_Par_Ref)

Instructions:

lea eax, [rdi+2+rsi]

- LEA is normally for address calculations, but here it works as an adder.
- It combines both additions: Int_1_Par_Val + 2 + Int_2_Par_Val.
- Result is stored in eax (32-bit register).

mov [rdx], eax

- Store the value from eax into the memory location pointed to by Int_Par_Ref.

ret

- Return from the function.

Arm_V8 (O3) Proc_7:

```
118 Proc_7:
119 .LFB51:
120     .cfi_startproc
121 // dry.c:906:    Int_Loc = Int_1_Par_Val + 2;
122     add w0, w0, 2   // Int_Loc, tmp97,
123 // dry.c:907:    *Int_Par_Ref = Int_2_Par_Val + Int_Loc;
124     add w0, w0, w1  // tmp96, Int_Loc, tmp98
125 // dry.c:907:    *Int_Par_Ref = Int_2_Par_Val + Int_Loc;
126     str w0, [x2]    // tmp96, *Int_Par_Ref_6(D)
127 // dry.c:908: } /* Proc_7 */
128     ret
129     .cfi_endproc
130 .LFE51:
131     .size   Proc_7, .-Proc_7
132     .align  2
133     .p2align 3,,7
134     .global Proc_8
135     .type   Proc_8, %function
136 Proc_8:
```

Explanation:

Different Registers:

- **w0** = first argument (Int_1_Par_Val)
- **w1** = second argument (Int_2_Par_Val)
- **x2** = third argument (Int_Par_Ref)

Instruction:

add w0, w0, 2

- Add the immediate value 2 to Int_1_Par_Val.
- Store the result back in w0.

add w0, w0, w1

- Add Int_2_Par_Val to the value in w0.
- Final result is now in w0.

str w0, [x2]

- Store the result from w0 into the memory location pointed to by Int_Par_Ref.

Unlike x86, **ARM does not merge both additions into one instruction. It uses two separate add instructions, which matches the RISC philosophy of simple, fixed-size operations.**